

Graph-Based Vulnerability Retrieval

Master's Thesis — Technical Progress · Checkpoint 4

Lívia

TU Munich · Siemens

May 4, 2026

Agenda

- 🔄 **Checkpoint 3 recap** — PCA & L2 ablation results
- 📊 **Embedding space analysis** — single-space & cross-space diagnostics
- 🔧 **Embedding fusion strategies** — 3-way combination & 4-way CodeBERT experiment
- 🤖 **Agent patching baseline** — LLM patch generation by analogy
- 🛡️ **LLM semantic evaluation** — GPT-4o as vulnerability auditor
- 💡 **Key findings & next steps**

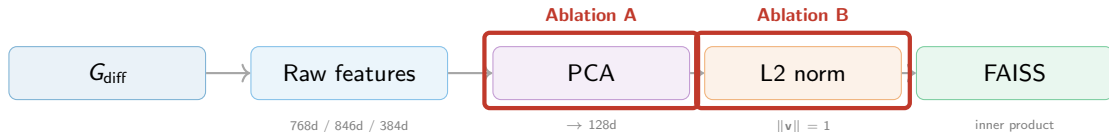
Checkpoint 3 Recap

Pipeline validation: PCA & L2 normalization

Last checkpoint established that the Combined embedder (NetLSD + WL + GIN) reaches 84.9% hit@1 with PCA + L2 normalization.

This week we investigate **why** and **whether the fusion can be improved**.

Combined Embedding Pipeline: Where PCA & L2 Normalization Sit



PCA (dimensionality reduction)

- Projects high-dim features to 128d
- Learns optimal linear basis from data
- Decorrelates and compresses — fuses heterogeneous streams (e.g. 78d structural + 768d CodeBERT)

L2 normalization (scale invariance)

- Maps every embedding onto the unit hypersphere
- FAISS inner product = cosine similarity
- Similarity depends on **direction** (pattern), not **magnitude** (graph size)

Ablation Results: PCA and L2 Normalization ($n=225$, self-retrieval)

Embedder	PCA + L2 (baseline)			No PCA, L2 only (Abl. A)			No PCA, no L2 (Abl. B)		
	hit@1	MRR	CWE	hit@1	MRR	CWE	hit@1	MRR	CWE
Combined	84.9%	.890	.501	74.0%	.780	.398	1.4%	.047	.071
CodeBERT seq	75.3%	.793	.412	76.7%	.799	.402	1.4%	.107	.160
GIN	72.6%	.788	.443	72.6%	.788	.443	1.4%	.044	.068
CodeBERT pat.	71.2%	.768	.414	75.3%	.788	.394	1.4%	.132	.201
WL	65.8%	.720	.423	67.1%	.723	.435	0.0%	.045	.065
RGCN	60.3%	.676	.372	67.1%	.733	.390	0.0%	.014	.060

Baseline: PCA + L2

- Combined dominates: 84.9%
- PCA is critical for Combined (−10.9 pp without)

Abl. A: No PCA

- CodeBERT stable (± 1 pp)
- GIN identical — already 128d
- Combined collapses: PCA fuses the 3 sub-embedders

Abl. B: No L2

- **Catastrophic failure** — all $< 2\%$
- Mean sim $> 100k$ (magnitude dominates)
- L2 norm is non-negotiable

Embedding Space Analysis: Single-Space Metrics

Per-embedder quality diagnostics — measured on 225 index embeddings

Metric	What it measures
Isotropy	Are embeddings spread uniformly across directions? Low $\rightarrow$ a few dimensions dominate $\rightarrow$ retrieval degrades.
Hubness	Do some points appear as neighbors of <i>everyone</i> ? High skewness $\rightarrow$ false-positive retrievals.
Alignment / Uniformity	Alignment : same-CWE pairs are close. Uniformity : all points spread on hypersphere. Decomposes retrieval quality into two axes.
Intra/Inter ratio	$\frac{\text{mean intra-CWE sim}}{\text{mean inter-CWE sim}}$. >1 means classes separated $\rightarrow$ good CWE recall.
Distance concentration	$\frac{d_{\max} - d_{\min}}{d_{\min}}$ per query. Low $\rightarrow$ all distances alike $\rightarrow$ cosine unreliable.

Embedding Space Analysis: Cross-Space Metrics

Cross-space comparison (Combined vs. individual embedders)

Metric	What it measures
Linear CKA	Structural similarity between two embedding matrices. Invariant to rotation/scaling. High $\rightarrow$ spaces encode the same geometry.
k -NN overlap	% shared neighbors between two spaces. High $\rightarrow$ same retrieval behavior (redundant). Low $\rightarrow$ complementary signal.
Rank correlation	Spearman ρ of pairwise distances. High $\rightarrow$ one embedder dominates the fusion.
Trustworthiness	Did PCA (384d $\rightarrow$ 128d) distort local neighborhoods? 1.0 = perfect preservation. Low $\rightarrow$ PCA causes retrieval errors.

Embedding Space Analysis: Experiment Design

Setup (same split as retrieval experiments)

- 225 index graphs (G_{diff}), seed=42, stratified
- 6 embedders: Combined, GIN, WL, CodeBERT seq, CodeBERT pat., RGCN
- All projected to 128d (PCA) + L2-normalized
- CWE labels as class signal (~ 15 unique classes)

Analysis structure (no grid — fixed dataset)

Analysis	Output
Per-embedder	Isotropy, hubness, alignment/uniformity, intra/inter, distance concentration
Pairwise matrices	6×6 CKA, k -NN overlap, rank correlation
Combined PCA	Trustworthiness at $k \in \{5, 10, 20\}$
Sensitivity	Hubness skewness vs. k (stability check)

Key questions this answers

- 1 **Does Combined add value?**
If $\text{CKA}(\text{Combined}, \text{GIN}) \approx 1 \rightarrow \text{Combined} \approx \text{GIN}$ with extra cost.
If k -NN overlap is low $\rightarrow$ Combined retrieves differently.
- 2 **Which sub-embedder dominates?**
Rank correlation identifies whose distance structure is preserved in the fusion.
- 3 **Is PCA lossy?**
Trustworthiness $< 0.9 \rightarrow$ PCA distorts neighborhoods $\rightarrow$ explains Combined-specific misses.
- 4 **Why do some CWEs fail?**
Per-CWE intra/inter ratio shows which classes overlap in each space.
- 5 **Are there hub points?**
Hub fraction $> 5\% \rightarrow$ some samples “attract” queries $\rightarrow$ systematic false positives.

Run: `python -m experiments.space_analysis_experiment`

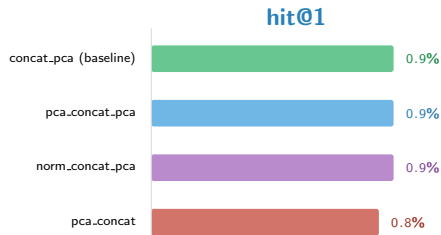
Combination Strategies: Experiment Design

Goal

The Combined embedder concatenates NetLSD + WL + GIN then reduces with PCA. Does pre-processing each sub-embedder *before* concatenation improve the fused space?

Strategies compared

Strategy	Pipeline
concat_pca	Concatenate raw $\rightarrow$ PCA to 128d (baseline)
pca_concat_pca	PCA each to 128d $\rightarrow$ concat (384d) $\rightarrow$ PCA to 128d
pca_concat	PCA each to 42d $\rightarrow$ concat to 126d (no 2 nd PCA)
norm_concat_pca	L2-norm each $\rightarrow$ concat $\rightarrow$ PCA to 128d



225 index, 73 query (G_{diff} , seed=42).
Each sub-embedder: 128d (NetLSD, WL, GIN).

Run: `python -m experiments.combining_experiment`

Combination Strategies: Results ($n=73$)

Strategy	Retrieval						Space Quality		
	hit@1	hit@5	hit@10	MRR	nDCG@10	MAP@10	Eff. Dim	Isotropy	Hub Skew
concat_pca	89.0%	95.9%	97.3%	.915	.687	.567	30.9	.0024	1.40
pca_concat_pca	89.0%	95.9%	97.3%	.915	.687	.567	30.9	.0023	1.40
norm_concat_pca	89.0%	95.9%	97.3%	.915	.687	.567	30.9	.0024	1.40
pca_concat	83.6%	95.9%	97.3%	.882	.665	.538	23.0	≈ 0	1.09

Combination Strategies: Analysis

Baseline is optimal

- Three strategies are **identical** to 4+ decimal places
- Sub-embedders already output 128d L2-normed vectors → pre-processing is a no-op

pca_concat hurts

- hit@1 drops -5.4 pp, MRR -3.3 pp
- PCA to 42d per embedder loses too much information
- Isotropy collapses (≈ 0): space degenerate

Why single PCA wins

- Final PCA over full 384d can exploit **cross-embedder correlations**
- Independent per-embedder PCA cannot discover joint structure
- Eff. dim 30.9 vs. 23.0: richer space

Combination Strategies: Key Findings

- ✓ **Current pipeline is already well-conditioned.** Pre-normalizing or pre-reducing individual embeddings before concatenation adds zero measurable benefit — the sub-embedders already produce 128d L2-normalized vectors. *No pipeline change needed.*
- ✗ **Double PCA without final reduction = information loss.** Reducing each embedder to 42d then concatenating (126d, no second PCA) drops hit@1 by 5.4 pp and collapses isotropy to ≈ 0 . *The final PCA is essential — it finds the best 128 directions across the full 384d concatenated space.*
- 📊 **Cross-embedder correlations matter.** A single PCA over the joint space (eff. dim 30.9) outperforms three independent PCAs (eff. dim 23.0) because it captures shared structure between NetLSD, WL, and GIN.
- 💡 **Implication for future embedders.** When adding new sub-embedders (e.g. CodeBERT), they should be concatenated raw into the joint space and let PCA handle fusion — no per-embedder preprocessing is warranted. *But does this actually work? → next experiment.*

4-Way Fusion: Motivation

Goal

Can adding a semantic signal (CodeBERT-pattern) to the structural fusion (NetLSD + WL + GIN) improve retrieval? We test two hypotheses: (1) raw concat lets PCA discover joint directions, (2) pre-PCA'ing CodeBERT to 128d equalizes scale.

Rationale

CodeBERT captures code *semantics*; structural embedders capture vulnerability *patterns*. Low CKA between them in prior experiments suggests complementary signals.

Run 1: raw concatenation

CodeBERT-pattern emits 802d (34 pattern + 768 CodeBERT). Concat with 3×128 d structural = 1186d $\rightarrow$ PCA to 128d.

Problem: CodeBERT contributes 68% of dimensions.

Run 2: balanced pre-PCA

PCA CodeBERT-pattern 802d $\rightarrow$ 128d first, then concat 4×128 d = 512d $\rightarrow$ PCA to 128d.

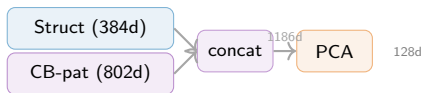
Each sub-embedder gets 25% of the pre-PCA space.

4-Way Fusion: Pipeline Comparison

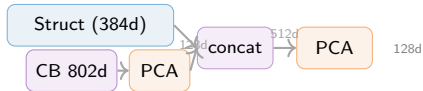
3-way (baseline)



4-way Run 1



4-way Run 2



4-Way Fusion: Results

Strategy	Retrieval				Space Quality			
	hit@1	hit@5	MRR	nDCG@10	Eff. Dim	Isotropy	Mean Sim	Intra/Inter
3-way baseline	90.4%	95.9%	.928	.690	31.2	.0023	.030	2.30
4-way raw concat	74.0%	86.3%	.786	.572	19.3	.0029	-.004	-6.46
(Run 1: 1186d→128d)								
4-way pre-PCA	74.0%	86.3%	.786	.572	19.2	.0028	-.004	-6.59
(Run 2: 512d→128d)								

4-Way Fusion: Analysis

CodeBERT hurts: -16 pp

- hit@1 drops from 90.4% to 74.0%
- MRR drops from .928 to .786
- Both runs produce **identical** degradation

Pre-PCA does not help

- Equalizing dimensions (512d) gives same result as raw (1186d)
- Problem is **signal conflict**, not scale imbalance
- CodeBERT dominates PCA regardless

Space explains the failure

- **Negative mean sim** (-0.004): embeddings anti-correlated
- **Negative intra/inter** (-6.5): class structure inverted
- Eff. dim 19 vs. 31: PCA captures wrong directions

4-Way Fusion: Key Findings

- ❌ **CodeBERT-pattern is incompatible with structural fusion.** Adding it degrades every metric by 15–16 pp. The semantic signal encodes *what code does*, not *how the vulnerability manifests structurally* — these are contradictory for CWE discrimination. *The signals destructively interfere in PCA.*
- ⚖️ **Dimension balancing is not the fix.** Pre-PCA'ing CodeBERT to 128d (equal to each structural embedder) produces identical results to raw 802d concatenation. PCA's top components still reflect CodeBERT's high-variance semantic directions, not vulnerability-discriminative structure.
- 📊 **Negative intra/inter ratio is the diagnostic.** Same-CWE pairs become *less* similar than cross-CWE pairs (−6.5 ratio). Two functions with the same CWE can have very different code semantics — CodeBERT sees them as “different” and overrides the structural “same-pattern” signal.
- 💡 **If CodeBERT is to help, it needs late fusion.** Concatenation + PCA is early fusion and fails here. Alternatives: separate retrieval pipelines with score-level combination (RRF), learned gating, or reranking with CodeBERT as a second stage.

Agent Baseline: Experiment Design

Goal

Given a vulnerable function and a retrieved similar CVE, can an LLM generate a correct patch **by analogy**?

Compare *oracle* retrieval (perfect) vs. *embedding* retrieval (our pipeline).

Model & prompt

- **GPT-4o** (Azure), temperature=0.2, max_tokens=4096
- **One-shot patching by analogy**: system prompt contains the retrieved CVE's fix-list, variable/function mapping, root cause
- Assistant message shows the example's CoT + patched code
- User message provides the target function to patch
- Agent must output [CoT START]...[Patched Code END]

Retrieval modes

Mode	What it retrieves
Oracle	Same-CVE pair from index (perfect match) — upper bound on agent quality
Embedding	Top-1 from Combined embedder + FAISS (realistic pipeline)

Evaluation protocol

- Same split: 225 index, 73 query (seed=42)
- **Patch quality**: BLEU-4, token Jaccard, CodeBLEU proxy
- **Retrieval**: hit@1, MRR, CWE hit@5
- **Confidence**: precision/recall at score thresholds

Agent Baseline: Results (Oracle, $n=73$, comments stripped)

CWE class	n	BLEU-4	Jaccard	CBP	BERTScore
Fixed Address to Pointer	1	.976	1.00	.963	.969
Resource Lifetime	1	.946	.938	.917	.934
Type Conversion	1	.943	.925	.882	.802
Memory Leak	2	.902	.981	.891	.911
Uninitialized Variable	2	.889	.951	.872	.841
Return of Stack Var	1	.888	.928	.847	.815
Check Return Value	3	.877	.847	.827	.904
Use After Free	12	.800	.860	.794	.861
Deadlock	4	.767	.801	.724	.767
Integer Overflow	10	.729	.766	.734	.699
Race Condition	5	.609	.802	.625	.703
Improper Locking	8	.340	.497	.390	.565
Expired Pointer Deref	2	.311	.354	.306	.587
Out-of-bounds Write	1	.175	.303	.245	.151
Overall	73	.655	.748	.660	.716

Agent Baseline: Effect of Comment Stripping

Oracle ($n=73$): with vs. without stripping

Metric	Raw	Stripped	Δ
BLEU-4	.511	.655	+144
ROUGE-L	.655	.766	+111
Jaccard	.561	.748	+187
CBP	.528	.660	+132
BERTScore	.619	.716	+097

Embedding ($n=73$): with vs. without stripping

Metric	Raw	Stripped	Δ
BLEU-4	.505	.637	+132
ROUGE-L	.636	.741	+105
Jaccard	.556	.731	+175
CBP	.516	.641	+125
BERTScore	.608	.691	+083

Why stripping matters

- The LLM typically **omits or rewrites** C/C++ comments (`//` and `/* */`)
- Comment differences are **not semantic errors** — they penalize all text-similarity metrics unfairly
- Stripping levels the comparison to **code-only** quality

What changes, what stays

- **All scores rise** 10–25 pp after stripping
- **Relative CWE ranking preserved:** Improper Locking and OOB Write remain hardest
- **Use After Free** jumps from .580 to **.800** BLEU-4 — most of the “error” was comment mismatch

```
Flag: python -m src.evaluate.evaluate_patches \
results.jsonl --strip-comments
```

Agent Baseline: Oracle vs. Embedding (BERTScore & ROUGE)

Overall comparison (comments stripped, $n=73$)

Metric	Oracle	Embed	$\Delta\%$
BERTScore	.716	.691	-3.5
ROUGE-1	.819	.791	-3.4
ROUGE-2	.743	.717	-3.5
ROUGE-L	.766	.741	-3.3

i Both runs now evaluate the same 73 queries. Embedding retrieval loses only **3–4%** across all metrics — patch generation quality is **nearly identical** regardless of retrieval mode.

Per-CWE highlights

↑ Embedding $\geq$ Oracle (6 CWEs):

- Array Index Validation (+23% BERT)
- Resource Exhaustion (+32% BERT)
- NULL Pointer Deref (+9% BERT)
- Race Condition (+1% BERT, +6% RL)

↓ Embedding $<$ Oracle (notable drops):

- Check Return Value (-33% BERT)
- Use After Free (-16% BERT)
- Type Conversion (-14% BERT)
- OOB Write (-95% BERT, $n=1$)

i Full per-CWE breakdown on next slide.

Agent Baseline: Per-CWE Oracle vs. Embedding (BERTScore & ROUGE-L)

CWE class	n	hit@1 (%)	Orac. BERT	Emb. BERT	$\Delta\%$	Orac. RG-L	Emb. RG-L	$\Delta\%$
Fixed Address to Pointer	1	100	.969	.969	0.0	.993	.993	0.0
Resource Lifetime	1	100	.934	.934	0.0	.951	.951	0.0
Memory Leak	2	100	.911	.902	-0.9	.926	.921	-0.6
Check Return Value	3	33	.904	.603	-33.3	.934	.613	-34.4
Use After Free	12	75	.861	.721	-16.2	.874	.748	-14.4
Uninitialized Variable	2	100	.841	.853	+1.4	.953	.961	+0.8
Return of Stack Var	1	100	.815	.820	+0.6	.883	.864	-2.1
Type Conversion	1	100	.802	.692	-13.6	.978	.978	0.0
Deadlock	4	100	.767	.726	-5.4	.833	.842	+1.1
Resource Expiration	2	50	.765	.725	-5.3	.854	.838	-1.8
Array Index Validation	1	0	.720	.882	+22.6	.745	.959	+28.7
Input Validation	1	0	.715	.715	0.0	.689	.689	0.0
Race Condition	5	100	.703	.712	+1.2	.689	.731	+6.1
Integer Overflow	10	60	.699	.705	+0.9	.829	.804	-3.1
Business Logic	1	100	.689	.750	+8.9	.702	.769	+9.6
Integer Underflow	1	100	.681	.726	+6.6	.840	.880	+4.7
NULL Pointer Deref	11	36	.629	.683	+8.6	.710	.723	+1.8
Expired Pointer Deref	2	100	.587	.583	-0.8	.466	.468	+0.5
Improper Locking	8	100	.565	.550	-2.6	.544	.525	-3.4
Resource Exhaustion	2	0	.539	.710	+31.7	.649	.665	+2.4
Unchecked Return Value	1	0	.428	.495	+15.6	.466	.603	+29.4
Out-of-bounds Write	1	0	.151	.008	-94.6	.454	.670	+47.6
Overall	73	68.5	.716	.691	-3.5	.766	.741	-3.3

📄 hit@1 = % queries where FAISS top-1 matches the correct CVE.
 ✅ 12/22 CWEs: embedding $\geq$ oracle
 ❌ Low hit@1 does **not** always

Retrieval: Combined vs. CodeBERT per CWE (hit@1)

CWE class	n	Comb.	CB-seq	CB-pat	GIN
OOB Write	1	0	0	0	0
Input Validation	1	0	100	100	0
Array Index Valid.	1	0	100	100	100
Check Return Value	3	33	0	0	33
NULL Pointer Deref	11	73	82	82	36
Integer Overflow	10	80	70	50	80
Use After Free	12	83	67	67	75
Improper Locking	8	88	100	75	75
Overall	73	82.2	75.3	71.2	72.6

CWEs with 100% hit@1 for all embedders omitted (12 rows).

CodeBERT rescues Combined's failures

- **Input Validation & Array Index:** Combined = 0%, both CodeBERTs = **100%** — semantic signal finds what structural cannot
- **NULL Pointer Deref** ($n=11$): CodeBERT 82% vs. Combined 73% — 1 extra query solved
- **Improper Locking** ($n=8$): CB-seq 100% vs. Combined 88%

Combined wins on high-volume CWEs

- **Use After Free** ($n=12$): Combined 83% vs. CodeBERT 67%
- **Integer Overflow** ($n=10$): Combined 80% vs. CB-pat 50%

💡 **Implication:** An ensemble (structural + semantic) could recover $\sim 3-4$ extra hits $\rightarrow$ push hit@1 from 82% toward $\sim 87\%$. This motivates late fusion / reranking.

Agent Baseline: Analysis & Failure Modes

Where the agent succeeds

- **Small, focused fixes** — adding overflow checks (`_builtin_add_overflow`), NULL guards, single-line resource cleanup
- **CWEs with stereotyped patterns**: Use After Free (BLEU .80), Uninitialized Variable (BLEU .89), Integer Overflow (BLEU .73)
- **DeepSeek-R1 variants** produce highest-quality patches (BLEU .775 vs. .586 for DeepSeek base)

Where the agent fails

- **Complex locking refactors** — multiple lock/unlock reorderings the LLM cannot reconstruct (BLEU .340)
- **Large multi-hunk diffs** — OOB Write (BLEU .175), agent cannot reproduce interleaved changes
- **Function-level mismatches** — Expired Pointer Deref (BLEU .311), agent generates structurally different code

LLM Semantic Evaluation: Experiment Design

Goal

Text-similarity metrics (BLEU, Jaccard) measure *surface overlap* with the ground truth, not whether the vulnerability is actually fixed. Can an LLM judge the **semantic correctness** of a patch?

Rationale

A patch can be textually different yet semantically correct (e.g. different variable names, reordered guards). Conversely, a high-BLEU patch may miss the critical fix. We need a **vulnerability-aware** evaluation.

Scope

All 73 oracle-mode patches from the agent baseline, evaluated with GPT-4o as a security auditor.

Prompt design

- 1 **Role:** “Expert security code auditor specializing in vulnerability analysis”
- 2 **Input:** CVE ID, CWE class, vulnerability description, original vulnerable code, generated patch
- 3 **Task:** Determine if the patch fixes the specific vulnerability described
- 4 **Output format:** structured JSON with `verdict` (FIXED / NOT_FIXED / PARTIAL), `confidence` (0.0–1.0), `reasoning` (free text)

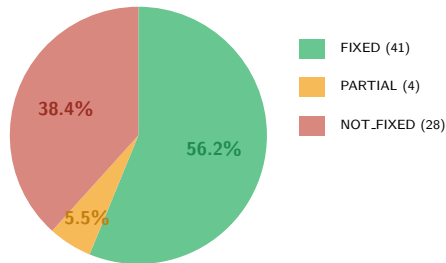
Model & parameters

- **GPT-4o** (Azure), temperature=0
- Deterministic evaluation (no sampling variance)
- 1.5s delay between calls (rate limiting)

LLM Semantic Evaluation: Overall Results ($n=73$)

Verdict	Count	%
FIXED	41	56.2%
PARTIAL	4	5.5%
NOT_FIXED	28	38.4%
Fix rate (FIXED + PARTIAL)	45	61.6%

Verdict	Avg. confidence
FIXED	0.948
NOT_FIXED	0.902
PARTIAL	0.750
Overall	0.919



Model: GPT-4o (Azure), temperature=0.

Run: `python -m src.evaluate.llm_evaluation results.jsonl`

LLM Semantic Evaluation: Per-CWE Breakdown

CWE class	n	FIXED	PARTIAL	NOT_FIXED	Fix rate
Array Index Validation	1	1	0	0	100%
Business Logic	1	1	0	0	100%
Input Validation	1	1	0	0	100%
Integer Underflow	1	1	0	0	100%
Out-of-bounds Write	1	1	0	0	100%
Resource Expiration	2	2	0	0	100%
Unchecked Return Value	1	1	0	0	100%
Uninitialized Variable	2	2	0	0	100%
Integer Overflow	10	7	2	1	90.0%
Use After Free	12	10	0	2	83.3%
Race Condition	5	3	0	2	60.0%
Deadlock	4	2	0	2	50.0%
Improper Locking	8	3	1	4	50.0%
Memory Leak	2	1	0	1	50.0%
Resource Exhaustion	2	1	0	1	50.0%
NULL Pointer Deref	11	3	1	7	36.4%
Check Return Value	3	1	0	2	33.3%
Expired Pointer Deref	2	0	0	2	0%
Fixed Address to Pointer	1	0	0	1	0%
Resource Lifetime	1	0	0	1	0%
Return of Stack Var	1	0	0	1	0%
Type Conversion	1	0	0	1	0%
Overall	73	41	4	28	61.6%

LLM Semantic Evaluation: Analysis vs. Text Metrics

Comparing evaluation signals

Metric	Score	Interpretation
hit@1 (retrieval)	89.0%	retrieves correct analogy
BLEU-4 (oracle, stripped)	.655	surface token overlap
ROUGE-L (oracle, stripped)	.766	longest common subseq.
BERTScore (oracle)	.716	contextual similarity
LLM fix rate	61.6%	vulnerability fixed?

i The gap matters: 89% retrieval → 61.6% semantic fix. The pipeline finds good analogies, but the agent fails to transfer the fix correctly ~38% of the time.

Where metrics disagree

- **High BLEU, NOT FIXED:** NULL Pointer Deref often produces “close” code that misses the critical NULL check placement — surface similarity masks semantic failure
- **Low BLEU, FIXED:** Some patches use different variable names or guard structures but correctly address the CVE — text metrics unfairly penalize valid alternatives
- **PARTIAL verdicts ($n=4$):** Patches address the vulnerability *partially* (e.g. add one of two required checks). Avg. confidence 0.75 vs. 0.95 for FIXED — the LLM is appropriately less certain

💡 Takeaway: LLM semantic evaluation captures patch correctness that text-similarity metrics miss. It should complement, not replace, automated metrics.

LLM Semantic Evaluation: Key Findings

- ✓ **61.6% of oracle patches semantically fix the CVE.** 41 fully fixed + 4 partial out of 73. High-confidence judgments (avg. 0.92) suggest the LLM is decisive, not guessing. *The agent produces meaningful patches for the majority of CWEs.*
- 📊 **CWE difficulty matches text-metric ranking.** Use After Free (83%) and Integer Overflow (90%) are the easiest — same CWEs where BLEU is highest. NULL Pointer Deref (36%) and Improper Locking (50%) remain hard. *Pattern stereotypy predicts both surface and semantic quality.*
- ⚠️ **Retrieval → generation gap: 89% → 62%.** The retrieval pipeline finds correct analogies for 89% of queries, but only 62% result in semantically correct patches. The bottleneck is the LLM's ability to **transfer** the fix pattern, not the retrieval quality.
- 💡 **LLM-as-judge complements text metrics.** It catches false positives (high BLEU but unfixed) and false negatives (low BLEU but correctly fixed). *Future evaluations should use both.*

Key Findings

- ✓ **PCA + L2 normalization is non-negotiable.** Without L2, all embedders collapse to $<2\%$ hit@1. Without PCA, Combined loses 10.9 pp. *The baseline pipeline is validated.*
- 📊 **3-way structural fusion is optimal.** Pre-processing sub-embedders before concatenation adds zero benefit. A single final PCA captures cross-embedder correlations (eff. dim 30.9 vs. 23.0).
- ✗ **Early fusion with CodeBERT fails (−16 pp).** Semantic and structural signals destructively interfere in PCA. CodeBERT needs late fusion (reranking, RRF, or gating).
- 🏠 **Embedding retrieval nearly matches oracle for patching.** Oracle BLEU .655 / Embedding .637 (−2.7%); ROUGE-L .766 / .741 (−3.3%). Stereotyped CWEs (Use After Free, Integer Overflow) patch well; complex locking refactors remain hard (.34).
- 🛡️ **LLM semantic evaluation: 61.6% of patches fix the CVE.** GPT-4o as security auditor confirms 41/73 fully fixed + 4 partial. Retrieval→generation gap (89%→62%) shows the bottleneck is fix transfer, not analogy retrieval.

Next Steps

-  **Analyse retrieval failure modes.** Embedding loses 14–16% on Use After Free — investigate which CVE variants are mis-retrieved and whether reranking helps.
-  **Late fusion with CodeBERT.** Test score-level RRF, learned gating, and CodeBERT-as-reranker to leverage semantic signal without PCA interference.
-  **Confusion analysis.** Characterize the 8.2% universal failure cases using CWE ontology Wu-Palmer distances — are failures taxonomically close or far?
-  **Multi-example prompting.** Retrieve top- k analogies instead of top-1 to give the agent more fix patterns for complex CWEs.

Repository: `graph-rag/` `python -m experiments.agent_experiment`